Machine Learning

AIML CZG565/ DSECLZG565

M5 : Decision Trees

BITS Pilani

Pilani Campus

innovate achieve lead

BITS Pilani

Contact Session 7,8

Course Plan

M1	Introduction
M2	Machine learning Workflow
M3	Linear Models for Regression
M4	Linear Models for Classification
M5	Decision Tree
M6	Instance Based Learning
M7	Support Vector Machine
M8	Bayesian Learning
M9	Ensemble Learning
M10	Unsupervised Learning
M11	Machine Learning Model Evaluation/Comparison

Work Integrated Learning Programmes

Module 5

- Information Theory
- Entropy Based Decision Tree Construction
- Avoiding Overfitting
- Minimum Description Length
- Handling Continuous valued attributes

Work Integrated Learning Programmes

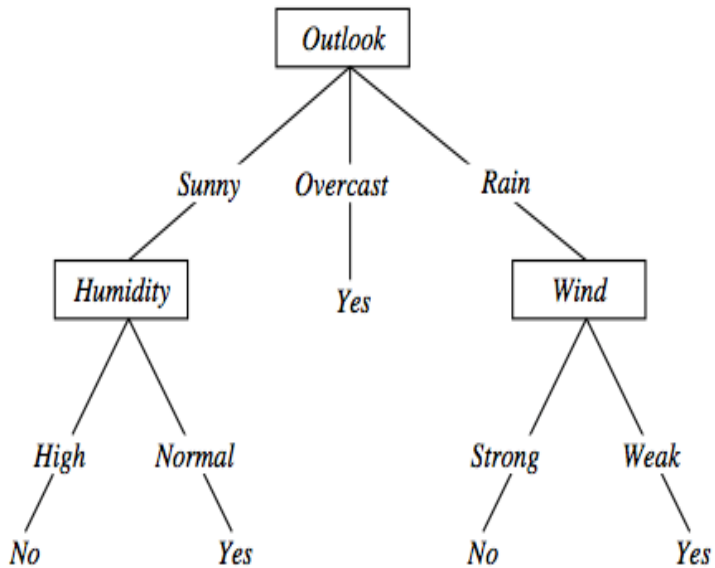
Decision Tree Classifier

Work Integrated Learning Programmes

Types of Classification

Decision Theory: Interpretation

Model Building

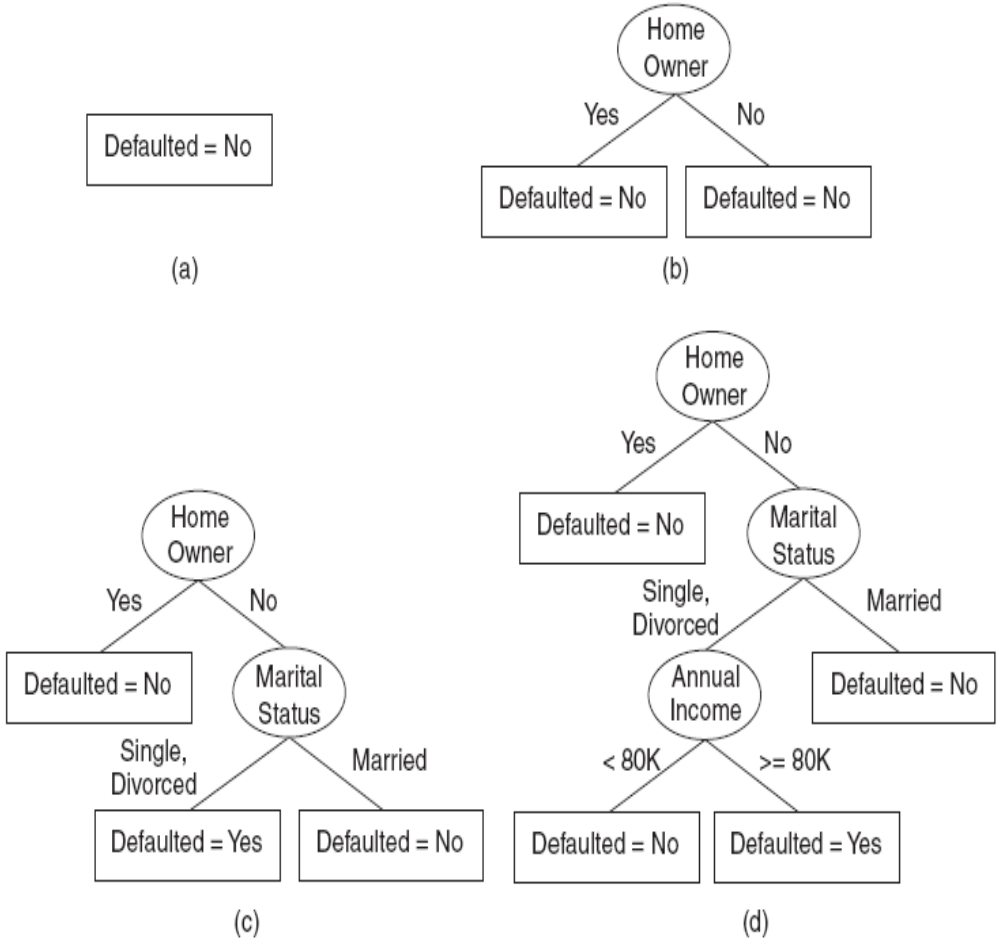


Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Logistic regression, SVMs , tree based classifiers (e.g. decision tree) Traditional neural networks, Nearest neighbor

Decision Tree Construction: Example

Tid	Home Owner (binary)	Marital Status (categorical)	Annual Income (continuous)	Defaulted Borrower (class)
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

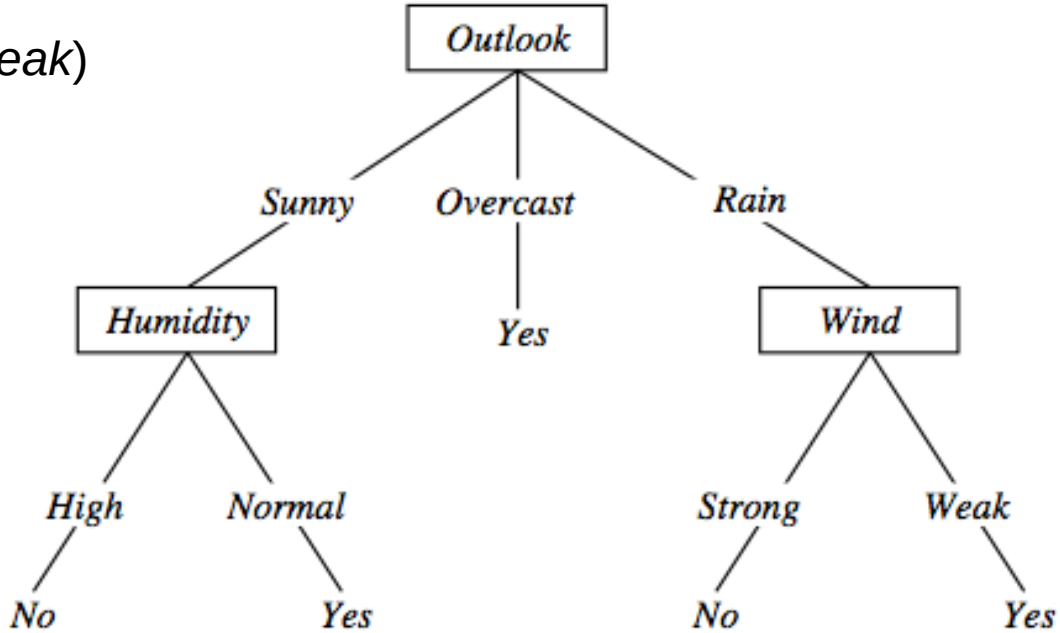


BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Decision Trees – Representation & Expressiveness

Decision trees represent a disjunction of conjunctions on constraints on the value of attributes:

(*Outlook* = *Sunny* ∧ *Humidity* = *Normal*) ∨
(*Outlook* = *Overcast*) ∨
(*Outlook* = *Rain* ∧ *Wind* = *Weak*)



$\langle Outlook=Sunny, Temp=Hot, Humidity=High, Wind=Strong \rangle$

No

Measure of Information

- The amount of information (surprise element) conveyed by a message is inversely proportional to its probability of occurrence. That is $I_k \, \alpha \, \frac{1}{p_k}$

- The mathematical operator satisfies above properties is the logarithmic operator. $I_k = \log_r \frac{1}{p_k} \; units$

Entropy

- Entropy of discrete random variable $X=\{x_1, x_2 \ldots x_n\}$

$$\mathrm{H}(X) = \mathrm{E}[\mathrm{I}(X)] = \mathrm{E}[-\log(\mathrm{P}(X))].$$

since: $\log_2(1/P(\text{event})) = -\log_2 P(\text{event})$

- As uncertainty increases, entropy increases
- Entropy across all values

$$\boxed{\mathrm{H}(X) = -\sum_{i=1}^{n} \mathrm{P}(x_i) \log_b \mathrm{P}(x_i)}$$

Entropy in general

- Entropy measures the amount of information in a random variable

$$H(X) = -p_+ \log_2 p_+ - p_- \log_2 p_- \qquad X = \{+, -\}$$

for binary classification [two-valued random variable]

$$H(X) = -\sum_{i=1}^{c} p_i \log_2 p_i = \sum_{i=1}^{c} p_i \log_2 1/p_i \qquad X = \{i, \ldots, c\}$$

for classification in *c* classes

Work Integrated Learning Programmes

Entropy in binary Classification

- Entropy measures the *impurity* of a collection of examples. It depends from the distribution of the random variable *p.*
  - *S* is a collection of training examples
  - p_+ the proportion of positive examples in *S*
  - p_- the proportion of negative examples in *S*

- $Entropy\ (S) \equiv -p_+ \log_2 p_+ - p_- \log_2 p_-$ $[0 \log_2 0 = 0]$
- $Entropy\ ([14+, 0–]) = -14/14 \log_2 (14/14) - 0 \log_2 (0) = 0$
- $Entropy\ ([9+, 5–]) = -9/14 \log_2 (9/14) - 5/14 \log_2 (5/14) = 0.94$
- $Entropy\ ([7+, 7–]) = -7/14 \log_2 (7/14) - 7/14 \log_2 (7/14) =$
- $= 1/2 + 1/2 = 1$ $[\log_2 1/2 = -1]$

Note: the log of a number < 1 is negative, $0 \le p \le 1$, $0 \le entropy \le 1$

https://www.easycalculation.com/log-base2-calculator.php

Decision Trees – Problem Type 1

Entropy of the Decision
Feature EnjoySport:

= **Entropy** $_{\textbf{DecisionClass}}$**(S) = Σ-p(c) log**$_2$ **p(c)**
= E(3,1) = – (3/4) $\log_2$ (3/4) – (1/4) $\log_2$ (1/4) = 0.4421+ 0.5 = 0.94

Sky	AirTemp	Humidity	Wind	Forecast	EnjoySport?
Sunny	Warm	Normal	Strong	Same	Yes
Sunny	Warm	High	Strong	Same	Yes
Rainy	Cold	High	Strong	Change	No
Sunny	Warm	High	Strong	Change	Yes
Rainy	Warm	Normal	Breeze	Same	Yes

Note:
- Class balancing is recommended before building
- Value = 0 to log base 2 k, where k is the number of classes you have.

Decision Trees – Problem Type 1

Sky	AirTemp	Humidity	Wind	Forecast	EnjoySport?
Sunny	Warm	Normal	Strong	Same	Yes
Sunny	Warm	High	Strong	Same	Yes
Rainy	Cold	High	Strong	Change	No
Sunny	Warm	High	Strong	Change	Yes

Entropy of the Decision Feature EnjoySport:

= **Entropy** $_{\text{DecisionClass}}$**(S) = Σ-p(c) log$_2$ p(c)**

= E(3,1) = – (3/4) $\log_2$ (3/4) – (1/4) $\log_2$ (1/4) = 0.4421+ 0.5 = 0.94

Sky	**Enjoy**	**Not Enjoy**
Sunny	3	0
Rainy	0	1

Entropy of the attribute Sky:

= p(Sunny) . E(3,0) + p(Rainy). E(0,1)

= (3/4) [– (3/3) $\log_2$ (3/3) – (0/3) $\log_2$ (0/3)] + (1/4) [– (0/1) $\log_2$ (0/1) – (1/1) $\log_2$ (1/1)]

= 0

Air Temp	**Enjoy**	**Not Enjoy**
Warm	3	0
Cold	0	1

Entropy of the attribute AirTemp:

= p(Warm) . E(3,0) + p(Cold). E(0,1)

= (3/4) [– (3/3) $\log_2$ (3/3) – (0/3) $\log_2$ (0/3)] + (1/4) [– (0/1) $\log_2$ (0/1) – (1/1) $\log_2$ (1/1)]

= 0

Decision Trees – Problem Type 1

Sky	AirTemp	Humidity	Wind	Forecast	EnjoySport?
Sunny	Warm	Normal	Strong	Same	Yes
Sunny	Warm	High	Strong	Same	Yes
Rainy	Cold	High	Strong	Change	No
Sunny	Warm	High	Strong	Change	Yes

Entropy of the Decision Feature EnjoySport:

= **Entropy $_{DecisionClass}$(S) = Σ-p(c) $\log_2$ p(c)**

= E(3,1) = – (3/4) $\log_2$ (3/4) – (1/4) $\log_2$ (1/4) = 0.94

Entropy of the attribute Humidity :

= p(Normal) . E(1,0) + p(High). E(2,1)

= (1/4) [– (1/1) $\log_2$ (1/1) – (0/1) $\log_2$ (0/1)] + (3/4) [– (2/3) $\log_2$ (2/3) – (1/3) $\log_2$ (1/3)]

= 0 + ¾ [0.39 + 0.52] = 0.68

Humidity	Enjoy	Not Enjoy
Normal	1	0
High	2	1

Entropy of the attribute Wind :

= p(Strong) . E(3,1)

= (4/4) [– (3/4) $\log_2$ (3/4) – (1/4) $\log_2$ (1/4)]

= 1(0.94) = 0.94

Wind	Enjoy	Not Enjoy
Strong	3	1

Decision Trees – Problem Type 1

Sky	AirTemp	Humidity	Wind	Forecast	EnjoySport?
Sunny	Warm	Normal	Strong	Same	Yes
Sunny	Warm	High	Strong	Same	Yes
Rainy	Cold	High	Strong	Change	No
Sunny	Warm	High	Strong	Change	Yes

Entropy of the Decision Feature EnjoySport:

= **Entropy** $_{\text{DecisionClass}}$**(S) = Σ-p(c) log$_2$ p(c)**

= E(3,1) = – (3/4) $\log_2$ (3/4) – (1/4) $\log_2$ (1/4) = 0.94

Entropy of the attribute Forecast :

= p(Same) . E(2,0) + p(Change). E(1,1)

= (2/4) [– (2/2) $\log_2$ (2/2) – (0/2) $\log_2$ (0/2)] + (2/4) [– (1/2) $\log_2$ (1/2) – (1/2) $\log_2$ (1/2)]

= 0 + 0.5 = 0.5

Forecast	Enjoy	Not Enjoy
Same	2	0
Change	1	1

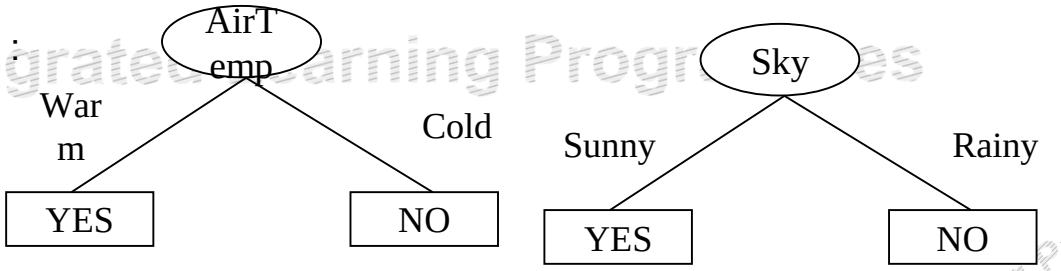
Decision Trees – Problem Type 1

Entropy of the Decision Feature EnjoySport: 0.94
Entropy of the attribute Sky: 0
Entropy of the attribute AirTemp: 0
Entropy of the attribute Humidity: 0.68
Entropy of the attribute Wind: 0.94
Entropy of the attribute Forecast: 0.5

Sky	AirTemp	Humidity	Wind	Forecast	EnjoySport?
Sunny	Warm	Normal	Strong	Same	Yes
Sunny	Warm	High	Strong	Same	Yes
Rainy	Cold	High	Strong	Change	No
Sunny	Warm	High	Strong	Change	Yes

Attribute that generalizes the model faster = That which reduces the randomness if used to decide = That which have the highest info gain = SKY & AIRTEMP

Decision Tree constructed
(both are equally correct decision tree) :

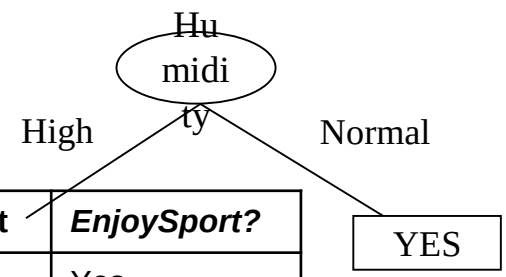


Decision Trees – Problem Type 2

(If there are heterogeneous nodes, continue the decision tree construction)

Sub Problem which has more scope to Model: None

Sky	AirTemp	Humidity	Wind	Forecast	EnjoySport?
Sunny	Warm	Normal	Strong	Same	Yes
Sunny	Warm	High	Strong	Same	Yes
Rainy	Cold	High	Strong	Change	No
Sunny	Warm	High	Strong	Change	Yes



Sky	**AirTemp**	**Wind**	**Forecast**	***EnjoySport?***
Sunny	Warm	Strong	Same	Yes
Rainy	Cold	Strong	Change	No
Sunny	Warm	Strong	Change	Yes

Assumed sample data in the node

Convergence Criteria :

Repeat the step again for the remaining features till:

- All training data are learnt
- No more learning is possible
- Constraints posed by the hyperparameter satisfied (More on this will be discussed in the next session under handling overfitting)

- *Information gain* is the *expected* reduction in entropy caused by partitioning the examples on an attribute.
- The higher the information gain the more effective the attribute in classifying training data.
- Expected reduction in entropy knowing *A*

$$Gain(S, A) = Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} Entropy(S_v)$$

Values(*A*) possible values for *A*

S_v subset of *S* for which *A* has value *v*

Maximizing the gain = minimizing the weighted average impurity measure of children nodes

Example: Information gain

Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

- Let
  - *Values*(*Wind*) = {*Weak*, *Strong*}
  - S = [9+, 5−]
  - S_{Weak} = [6+, 2−]
  - S_{Strong} = [3+, 3−]
- Information gain due to knowing *Wind*:

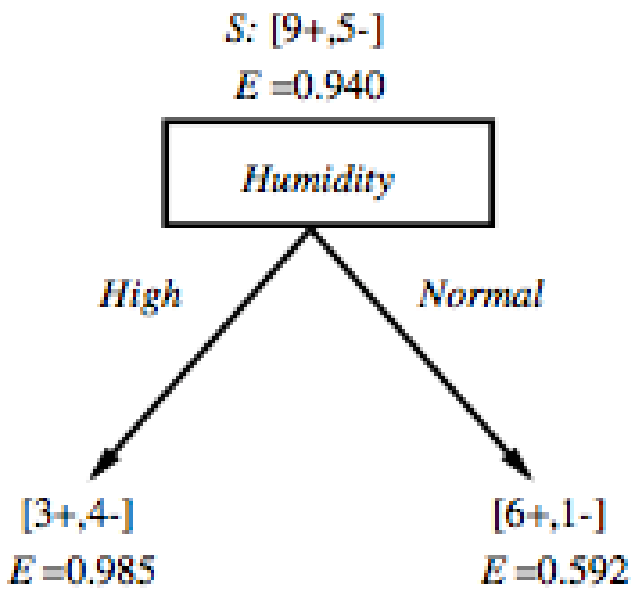
$$Gain(S, Wind) = Entropy(S) - 8/14\ Entropy(S_{Weak}) - 6/14\ Entropy(S_{Strong})$$

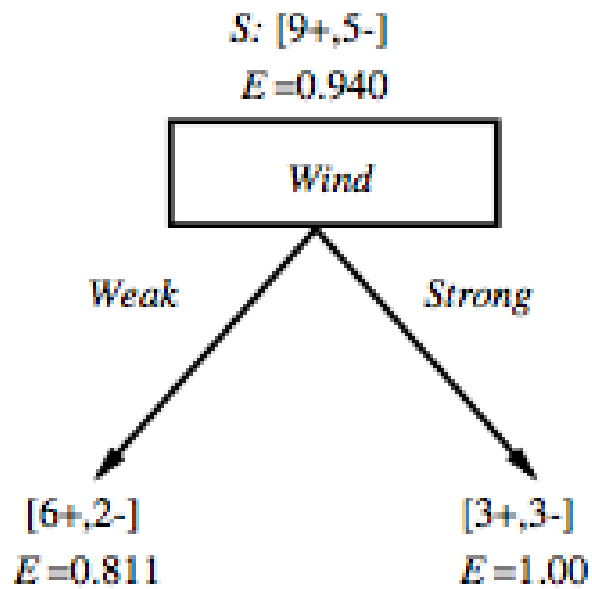
$$= 0.94 - 8/14 \times 0.811 - 6/14 \times 1.00$$

$$= 0.048$$

Which attribute is the best classifier?

Which attribute is the best classifier?





Gain (S, Humidity)

= .940 - (7/14).985 - (7/14).592
= .151

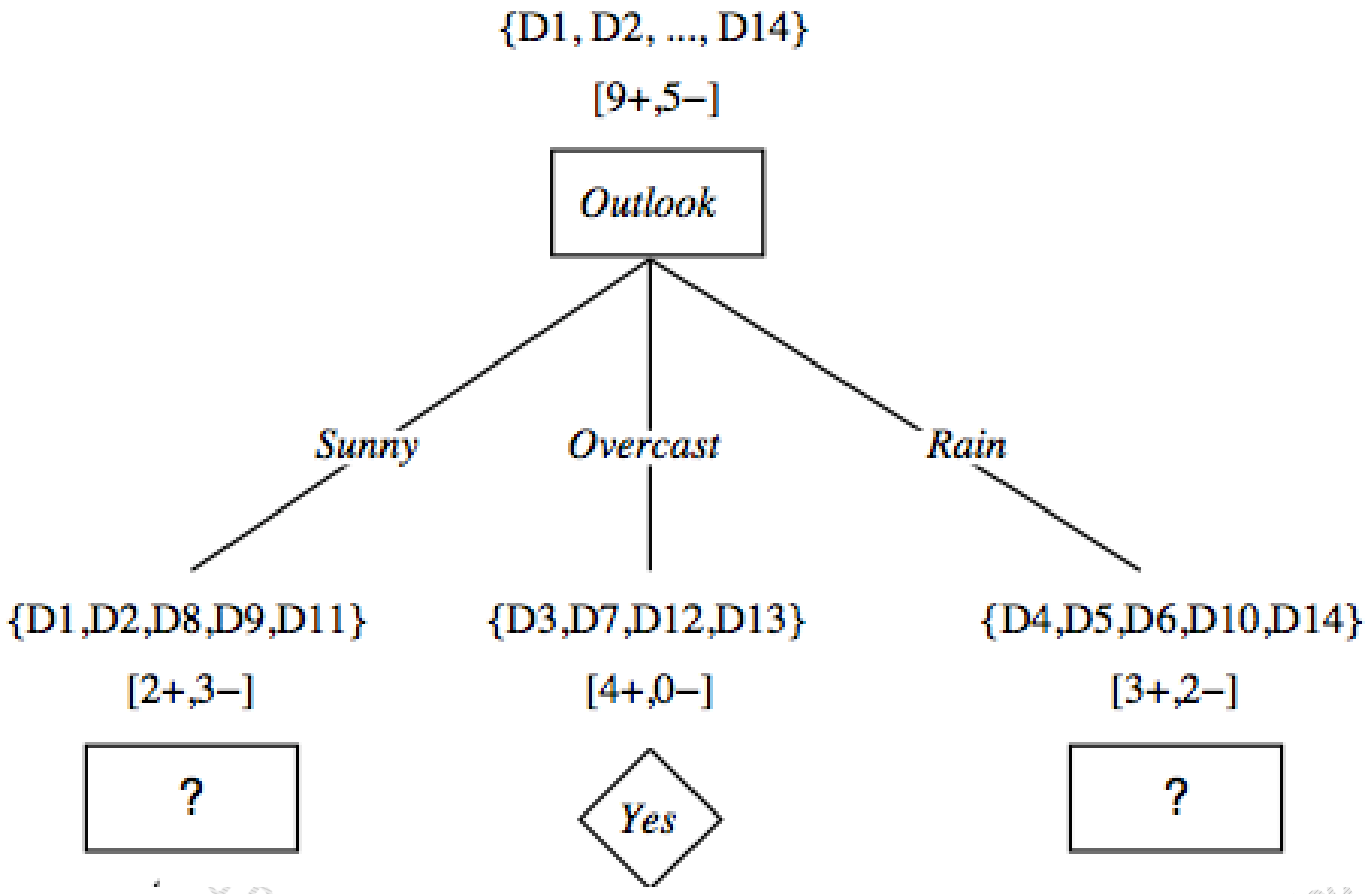
Gain (S, Wind)

= .940 - (8/14).811 - (6/14)1.0
= .048

First step: which attribute to test at the root?

- Which attribute should be tested at the root?
  - *Gain*(*S*, *Outlook*) = **0.246**
  - *Gain*(*S*, *Humidity*) = 0.151
  - *Gain*(*S*, *Wind*) = 0.084
  - *Gain*(*S*, *Temperature*) = 0.029
- ***Outlook*** provides the best prediction for the target
- Lets grow the tree:
  - add to the tree a successor for each possible value of *Outlook*
  - partition the training samples according to the value of *Outlook*

After first step



Second step

- Working on *Outlook=Sunny* node:

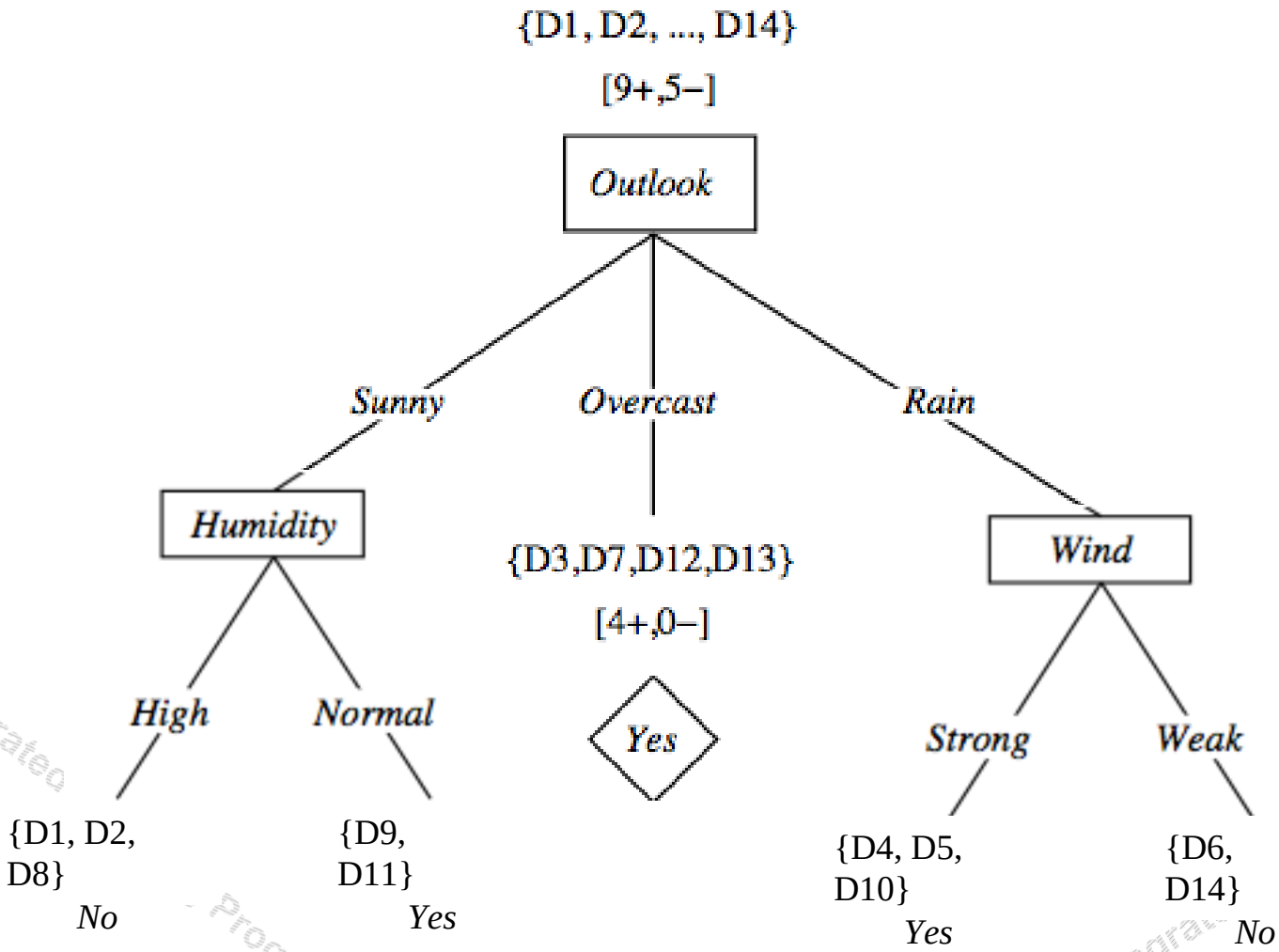
  $Gain(S_{Sunny}, Humidity) = 0.970 - 3/5 \times 0.0 - 2/5 \times 0.0 = \mathbf{0.970}$

  $Gain(S_{Sunny}, Wind) = 0.970 - 2/5 \times 1.0 - 3.5 \times 0.918 = 0\ .019$

  $Gain(S_{Sunny}, Temp.) = 0.970 - 2/5 \times 0.0 - 2/5 \times 1.0 - 1/5 \times 0.0 = 0.570$

- ***Humidity*** provides the best prediction for the target
- Lets grow the tree:
  - add to the tree a successor for each possible value of *Humidity*
  - partition the training samples according to the value of *Humidity*

Second and third steps



Convergence of Algorithm

- Conditions for stopping partitioning
  - All samples for a given node belong to the same class
  - There are no remaining attributes for further partitioning – majority voting is employed for classifying the leaf
  - There are no samples left

Work Integrated Learning Programmes

ID3: algorithm -Summary

ID3(*X, T, Attrs*) *X*: training examples:
T: target attribute (e.g. *PlayTennis*),
Attrs: other attributes, initially all attributes

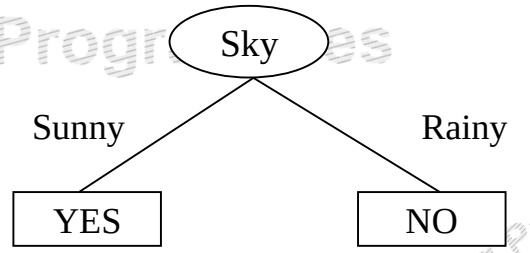
Create *Root* node
If all X's are +, *return* *Root* with class +
If all X's are –, *return* *Root* with class –
If *Attrs* is empty *return* *Root* with class most common value of *T* in *X*
else
$A \leftarrow$ best attribute; decision attribute for Root $\leftarrow A$
For each possible value v_i of *A:*
- add a new branch below *Root,* for test $A = v_i$
- $X_i \leftarrow$ subset of *X* with $A = v_i$
- *If* X_i is empty *then* add a new leaf with class the most common value of *T* in *X*
else add the subtree generated by ID3(X_i, *T, Attrs* − {*A*})

return *Root*

Decision Trees – Problem Type 3

Predict the class for new input instance □

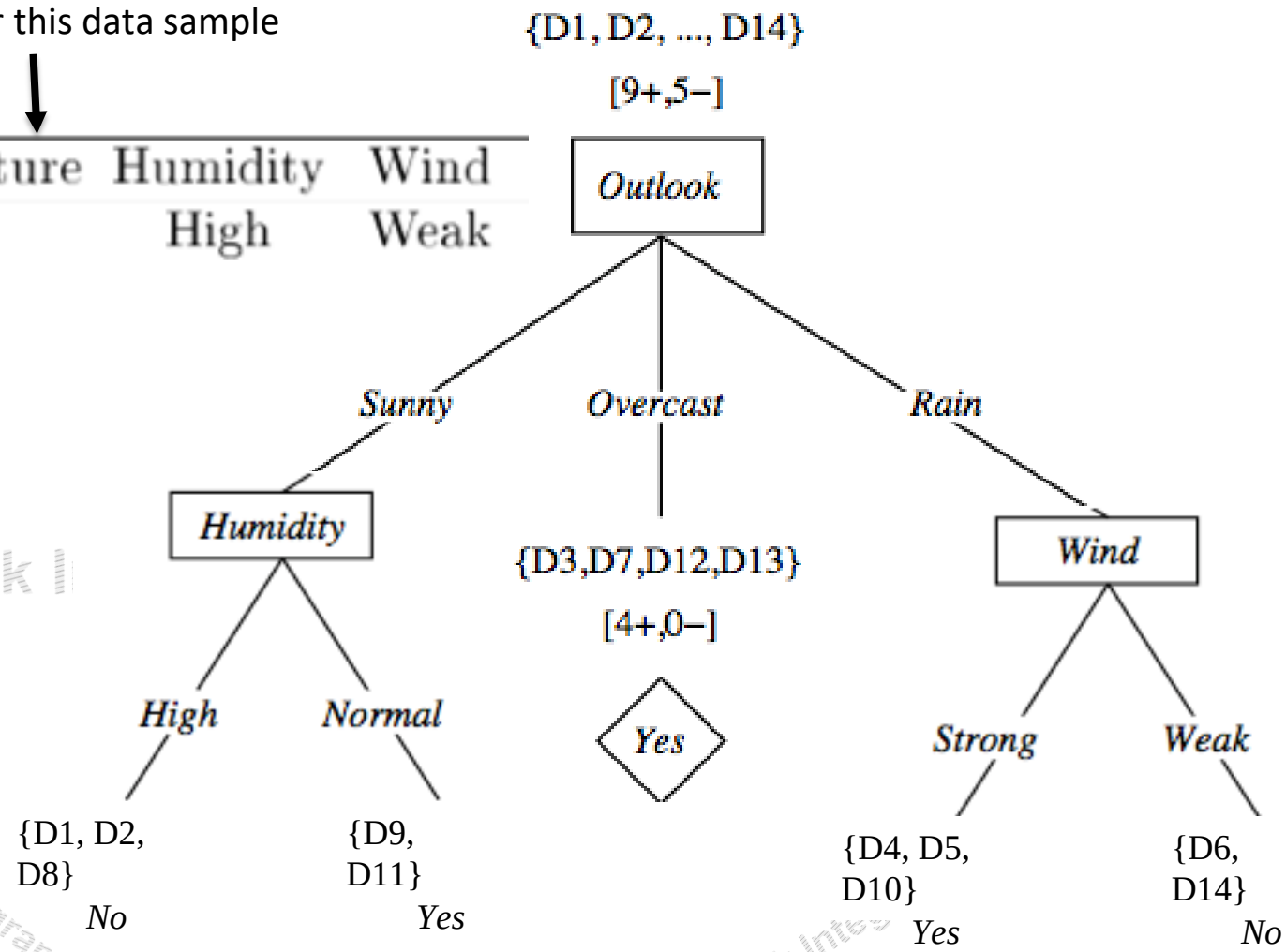
Sky	AirTemp	Humidity	Wind	Forecast	EnjoySport?
Sunny	Warm	Normal	Strong	Same	Yes
Sunny	Warm	High	Strong	Same	Yes
Rainy	Cold	High	Strong	Change	No
Sunny	Warm	High	Strong	Change	Yes
Rainy	Warm	Normal	Breeze	Same	????



Decision Trees – Problem Type 3

Predict the output for this data sample

Outlook	Temperature	Humidity	Wind
Sunny	Hot	High	Weak



Problem Type 4 – Evaluate the model

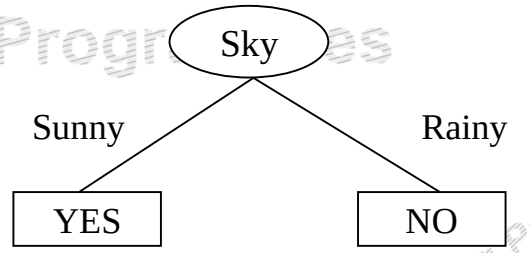
Confusion Matrix

Actual class\Predicted class	C_1	$\neg C_1$
C_1	**True Positives (TP)**	**False Negatives (FN)**
$\neg C_1$	**False Positives (FP)**	**True Negatives (TN)**

- The evaluation discussed under module 4 is applicable here
- By default , model's predictive quality is evaluated **only by the unseen validation/test instances.**
- When training performance (eg., accuracy) is required for comparison, the training instances are used for assessment

- $CM_{i,j}$ indicates the number of tuples of class i that were labelled by the classifier as class j.

Sky	AirTemp	Humidity	Wind	Forecast	EnjoySport?
Sunny	Warm	Normal	Strong	Same	Yes
Sunny	Warm	High	Strong	Same	Yes
Rainy	Cold	High	Strong	Change	No
Sunny	Warm	High	Strong	Change	Yes
Rainy	Warm	Normal	Breeze	Same	Yes



Decision trees

- Advantages:
  - Inexpensive to construct
  - Extremely fast at classifying unknown records
  - Easy to interpret for small-sized trees
  - Can easily handle redundant or irrelevant attributes (unless the attributes are interacting)
- Disadvantages:
  - Space of possible decision trees is exponentially large.
  - -- Greedy approaches are often unable to find the best tree.
  - Does not take into account interactions between attributes
  - Each decision boundary involves only a single attribute

Applications

Suited for following classification problems:

- Applications whose Instances are represented by attribute-value pairs.
- The target function has discrete output values
- Disjunctive descriptions may be required
- The training data may contain missing attribute values

Real world applications

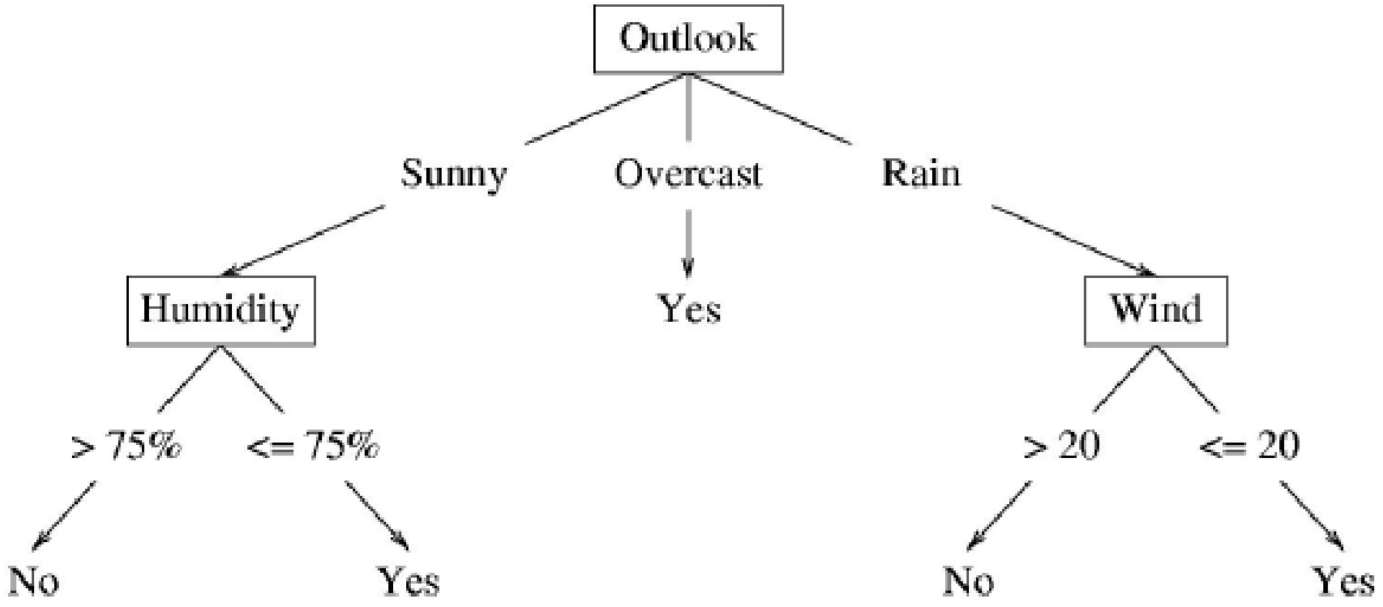
- Biomedical applications
- Manufacturing
- Banking sector
- Make-Buy decisions

Agenda

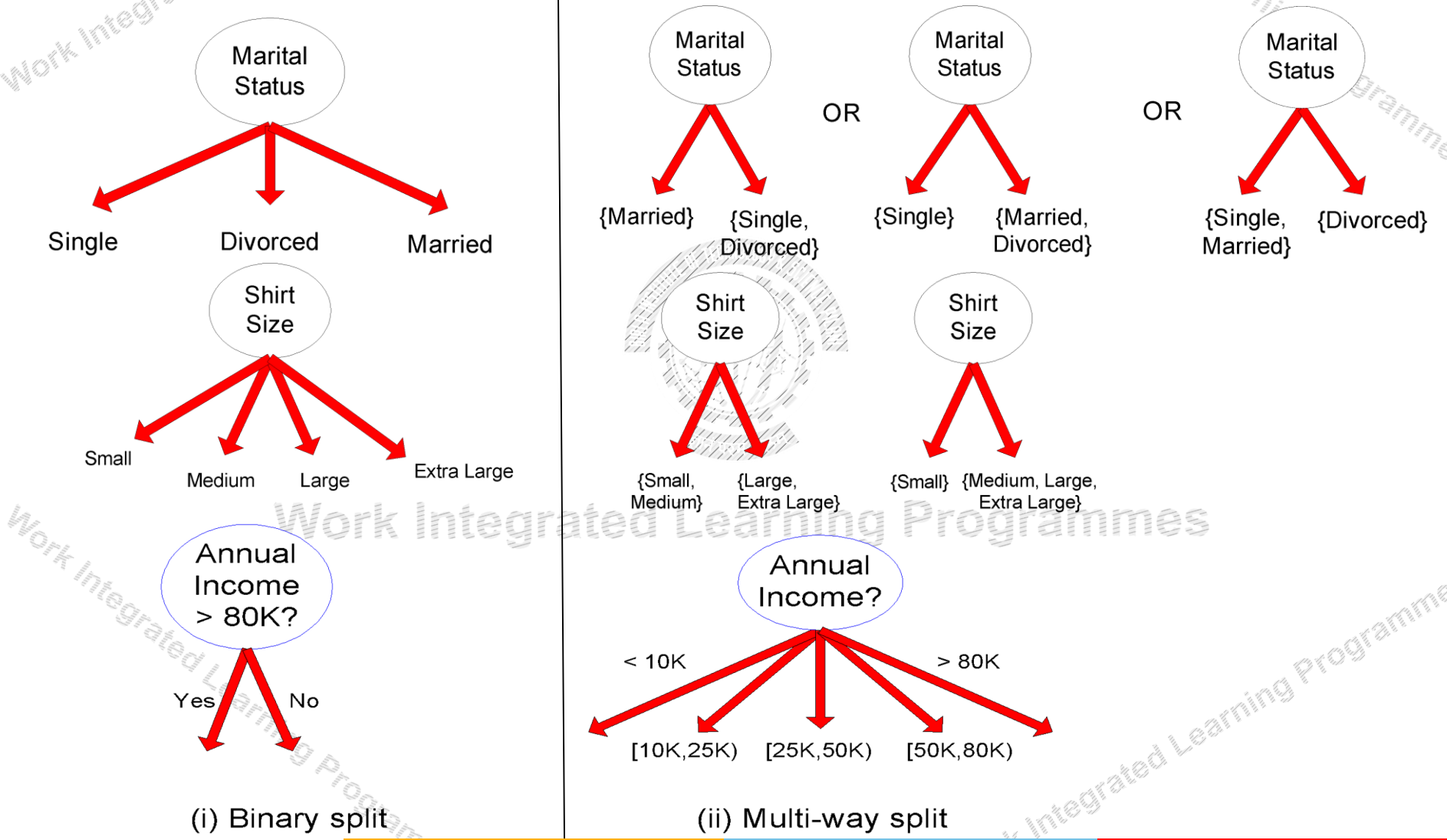
- Information Theory
- Entropy Based Decision Tree Construction
- Avoiding Overfitting
- Minimum Description Length
- Handling Continuous valued attributes

Work Integrated Learning Programmes

Decision Trees – Dealing with Continuous Values



Sample splits of Different Attribute Type



Decision Trees – Dealing with Continuous Values – Problem Type 5

- Given a continuous-valued attribute *A*, dynamically create a new attribute A_c

$$A_c = \text{True } \textit{if } A < c, \text{ False } \textit{otherwise}$$

- How to determine threshold value *c* ?

Way 1: Multi-way Split

- Example. *Temperature* in the *PlayTennis* example
  - Sort the examples according to *Temperature*

Temperature	40	48	|	60	72		80	|	90
PlayTennis	No	No	*54*	Yes	Yes		Yes	*85*	No

  - Determine candidate thresholds by averaging consecutive values where there is a change in classification: (48+60)/2=54 and (80+90)/2=85

Decision Trees – Dealing with Continuous Values

- Given a continuous-valued attribute *A*, dynamically create a new attribute A_c

$$A_c = \text{True } \textit{if } A < c, \text{ False } \textit{otherwise}$$

- How to determine threshold value *c* ?

Way 2: Binary Splits

- Example. *Annual Income*

Sort the examples according to *Annual Income*

Linearly scan all possible threshold, to determine best split point where the impurity is the least

ID	Home Owner	Marital Status	Annual Income	Defaulted?
1	Yes	Single	125000	No
2	No	Married	100000	No
3	No	Single	70000	No
4	Yes	Married	120000	No
5	No	Divorced	95000	Yes
6	No	Single	60000	No
7	Yes	Divorced	220000	No
8	No	Single	85000	Yes
9	No	Married	75000	No
10	No	Single	90000	Yes

Decision Trees – Dealing with Continuous Values – Problem Type 6

- Given a continuous-valued attribute A, dynamically create a new attribute A_c

$$A_c = \text{True } \textit{if } A < c, \text{ False } \textit{otherwise}$$

- How to determine threshold value c ?

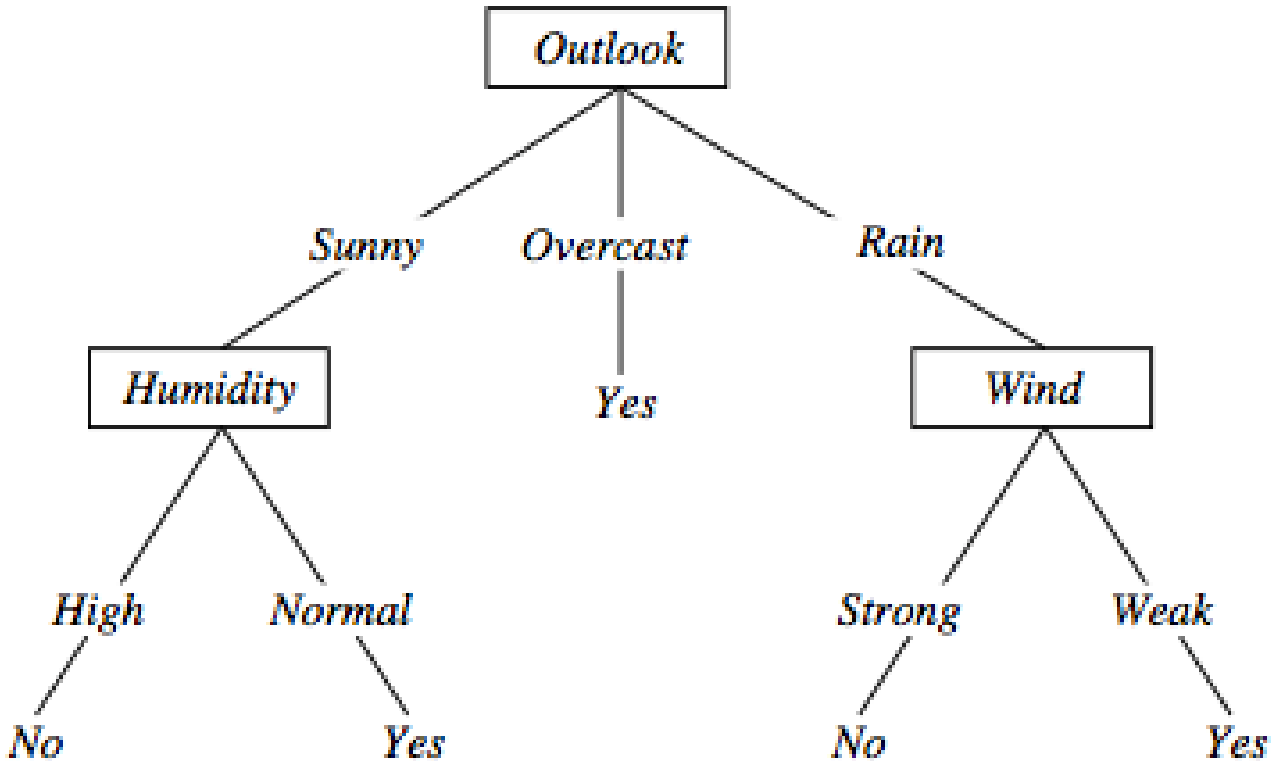
Way 2: Binary Splits

- Example. *Annual Income*

Class	No	No	No	Yes	Yes	Yes	No	No	No	No
	Annual Income (in '000s)									
Sorted Values →	60	70	75	85	90	95	100	120	125	220

Split Positions →	55		65		72.5		80		87.5		92.5		97.5		110		122.5		172.5		230	
	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>
Yes	0	3	0	3	0	3	0	3	1	2	2	1	3	0	3	0	3	0	3	0	3	0
No	0	7	1	6	2	5	3	4	3	4	3	4	3	4	4	3	5	2	6	1	7	0
Gini	0.420		0.400		0.375		0.343		0.417		0.400		***0.300***		0.343		0.375		0.400		0.420	

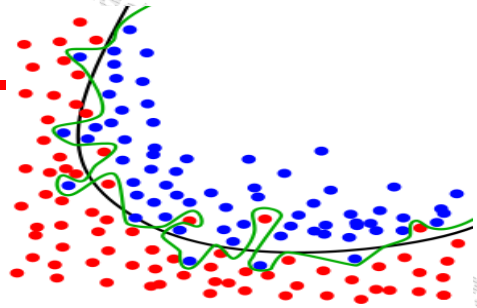
Overfitting in decision trees



⟨*Outlook=Sunny, Temp=Hot, Humidity=Normal, Wind=Strong, PlayTennis=No* ⟩

New noisy example causes splitting of second leaf node.

Overfitting and Tree Pruning



- Overfitting: An induced tree may overfit the training data
  - Too many branches, may reflect anomalies due to noise or outliers in the training data
  - Poor accuracy for unseen samples

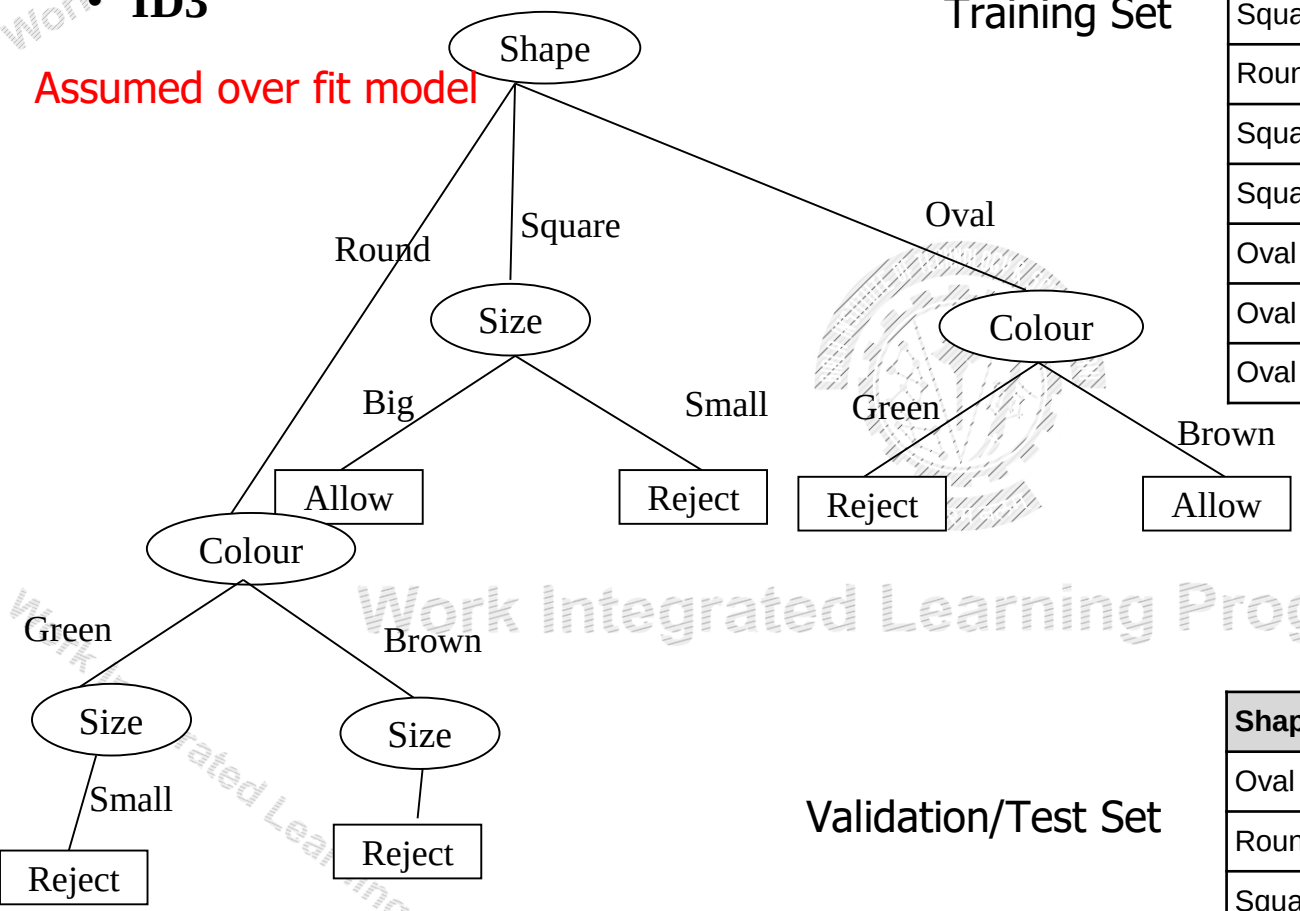
<u>Approaches to tackle:</u>

- Pruning
- Constraint on the depth of tree
- Constraint on the minimum nodes allowed at leaf

Over fitting

- **ID3**

Assumed over fit model



Training Set

Shape	Colour	Size	Action
Round	Green	Small	Reject
Square	Black	Big	Allow
Square	Brown	Big	Allow
Round	Brown	Small	Reject
Square	Green	Big	Allow
Square	Brown	Small	Reject
Oval	Green	Big	Reject
Oval	Brown	Small	Allow
Oval	Green	Small	Reject

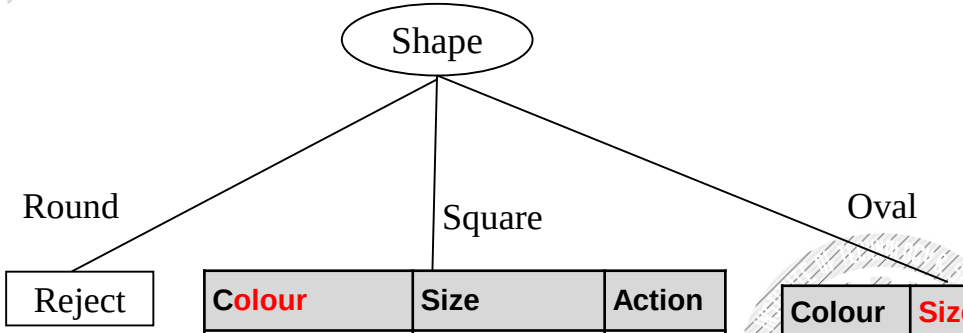
Validation/Test Set

Shape	Colour	Size	Action
Oval	Black	Small	Reject
Round	Brown	Big	Allow
Square	Brown	Big	Allow
Oval	Green	Small	Allow

Pre-Pruning – Problem Type 7

Shape	Colour	Size	Action
Round	Green	Small	Reject
Square	Black	Big	Allow
Square	Brown	Big	Allow
Round	Brown	Small	Reject
Square	Green	Big	Allow
Square	Brown	Small	Reject
Oval	Green	Big	Reject
Oval	Brown	Small	Allow
Oval	Green	Small	Reject

- *Threshold: Gain <=0.85*



Colour	Size	Action
Black	Big	Allow
Brown	Big	Allow
Green	Big	Allow
Brown	Small	Reject

Gain =0.8113

Colour	Size	Action
Green	Big	Reject
Brown	Small	Allow
Green	Small	Reject

Gain =0.9182

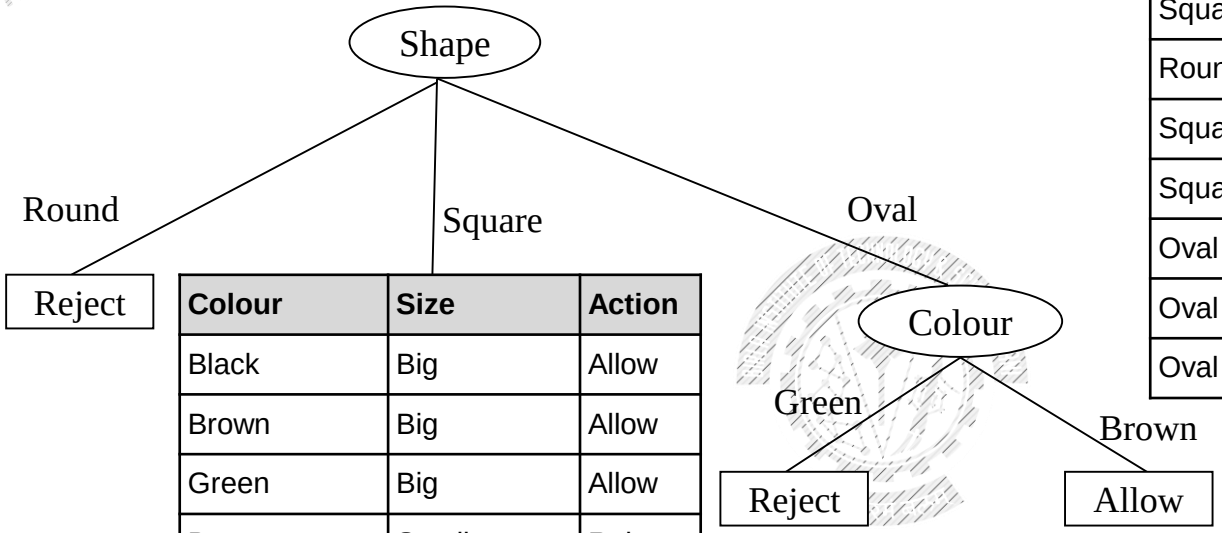
Shape	Colour	Size	Action
Oval	Black	Small	Reject
Round	Brown	Big	Allow
Square	Brown	Big	Allow
Oval	Green	Small	Allow

Idea : While construction Prune the nodes whose gain is greater than the predefined threshold

Pre-Pruning – Problem Type 7

Shape	Colour	Size	Action
Round	Green	Small	Reject
Square	Black	Big	Allow
Square	Brown	Big	Allow
Round	Brown	Small	Reject
Square	Green	Big	Allow
Square	Brown	Small	Reject
Oval	Green	Big	Reject
Oval	Brown	Small	Allow
Oval	Green	Small	Reject

- *Threshold: Gain <=0.85*



Colour	Size	Action
Black	Big	Allow
Brown	Big	Allow
Green	Big	Allow
Brown	Small	Reject

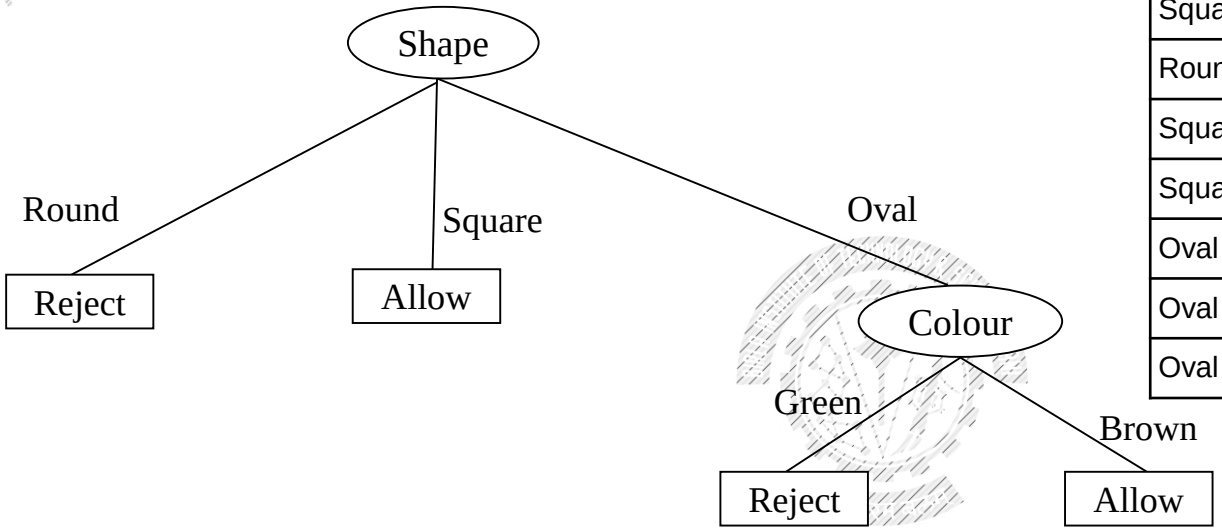
Shape	Colour	Size	Action
Oval	Black	Small	Reject
Round	Brown	Big	Allow
Square	Brown	Big	Allow
Oval	Green	Small	Allow

Note:

- DT can estimate the probability of instance's membership to a class

Pre-Pruning – Problem Type 7

- *Threshold: Gain <=0.85*



Shape	Colour	Size	Action
Round	Green	Small	Reject
Square	Black	Big	Allow
Square	Brown	Big	Allow
Round	Brown	Small	Reject
Square	Green	Big	Allow
Square	Brown	Small	Reject
Oval	Green	Big	Reject
Oval	Brown	Small	Allow
Oval	Green	Small	Reject

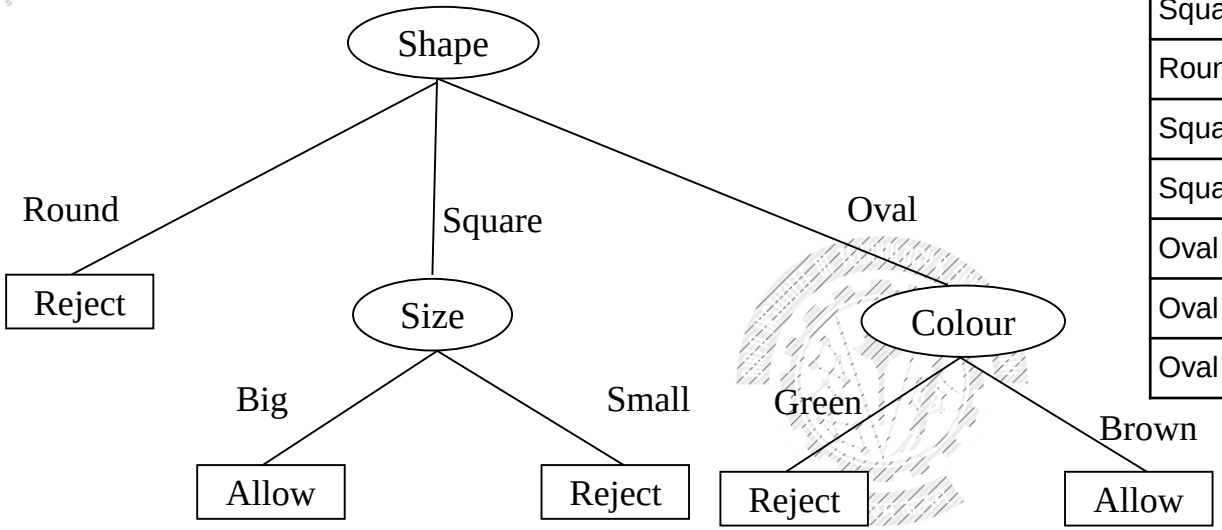
Apply majority voting for converting the pruned subset of data into a class

Shape	Colour	Size	Action
Oval	Black	Small	Reject
Round	Brown	Big	Allow
Square	Brown	Big	Allow
Oval	Green	Small	Allow

Work Integrated Learning Programmes

Post-Pruning – Problem Type 8

Assumed model from the output of one intermediate iteration of the decision tree building



Shape	Colour	Size	Action
Round	Green	Small	Reject
Square	Black	Big	Allow
Square	Brown	Big	Allow
Round	Brown	Small	Reject
Square	Green	Big	Allow
Square	Brown	Small	Reject
Oval	Green	Big	Reject
Oval	Brown	Small	Allow
Oval	Green	Small	Reject

Idea :

1. Post construction, scan the tree bottom-up
2. At every decision node

   Retain the attribute node & evaluate it against the prune set (validation set)

   Remove the attribute node & reevaluate it with the same prune set

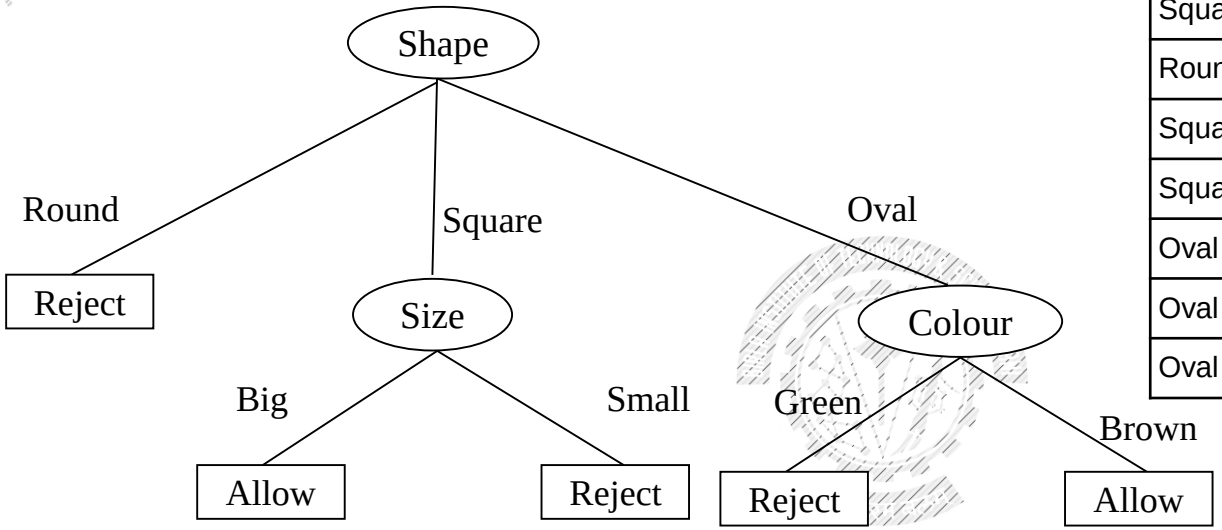
   If there is a reduction in error, prune the node else retain the node
3. Repeat this in other branches of the tree

Prune Set

Shape	Colour	Size	Action
Oval	Black	Small	Reject
Round	Brown	Big	Allow
Square	Brown	Big	Allow
Oval	Green	Small	Allow

Post-Pruning – Problem Type 8

Assumed model from the output of one intermediate iteration of the decision tree building



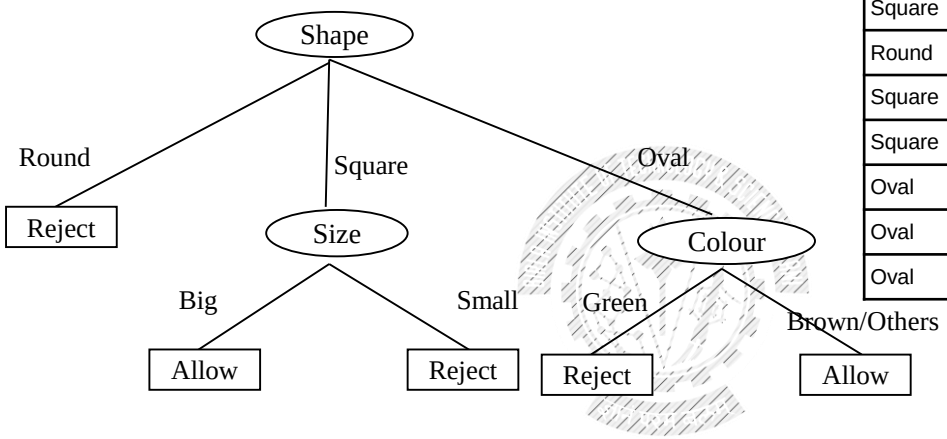
Shape	Colour	Size	Action
Round	Green	Small	Reject
Square	Black	Big	Allow
Square	Brown	Big	Allow
Round	Brown	Small	Reject
Square	Green	Big	Allow
Square	Brown	Small	Reject
Oval	Green	Big	Reject
Oval	Brown	Small	Allow
Oval	Green	Small	Reject

- Error rate is the percentage of tuples misclassified
- Prune set is used to estimate the cost

Prune Set

Shape	Colour	Size	Action
Oval	Black	Small	Reject
Round	Brown	Big	Allow
Square	Brown	Big	Allow
Oval	Green	Small	Allow

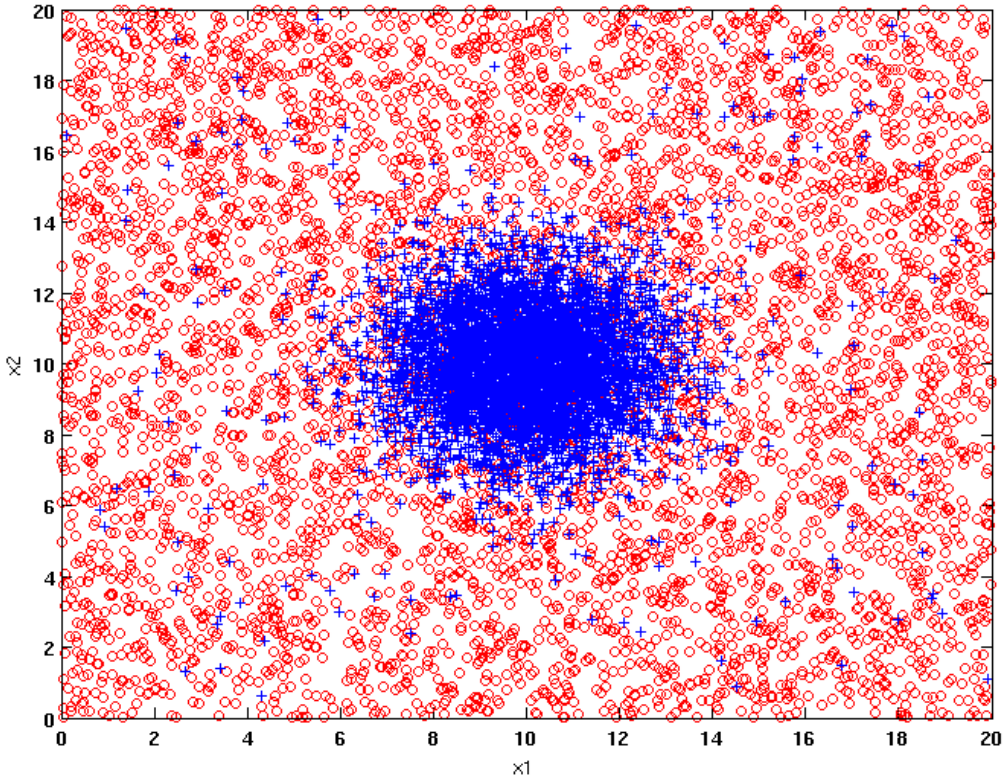
Post-Pruning – Problem Type 8



Shape	Colour	Size	Action
Round	Green	Small	Reject
Square	Black	Big	Allow
Square	Brown	Big	Allow
Round	Brown	Small	Reject
Square	Green	Big	Allow
Square	Brown	Small	Reject
Oval	Green	Big	Reject
Oval	Brown	Small	Allow
Oval	Green	Small	Reject

Prune Size &Predict	Prune Colour & Predict	Above Tree's Prediction	Shape	Colour	Size	Action
Allow	Reject	Allow	Oval	Black	Small	Reject
Reject	Reject	Reject	Round	Brown	Big	Allow
Allow	Allow	Allow	Square	Brown	Big	Allow
Reject	Reject	Reject	Oval	Green	Small	Allow

Example Data Set



Two class problem:

+ : 5200 instances

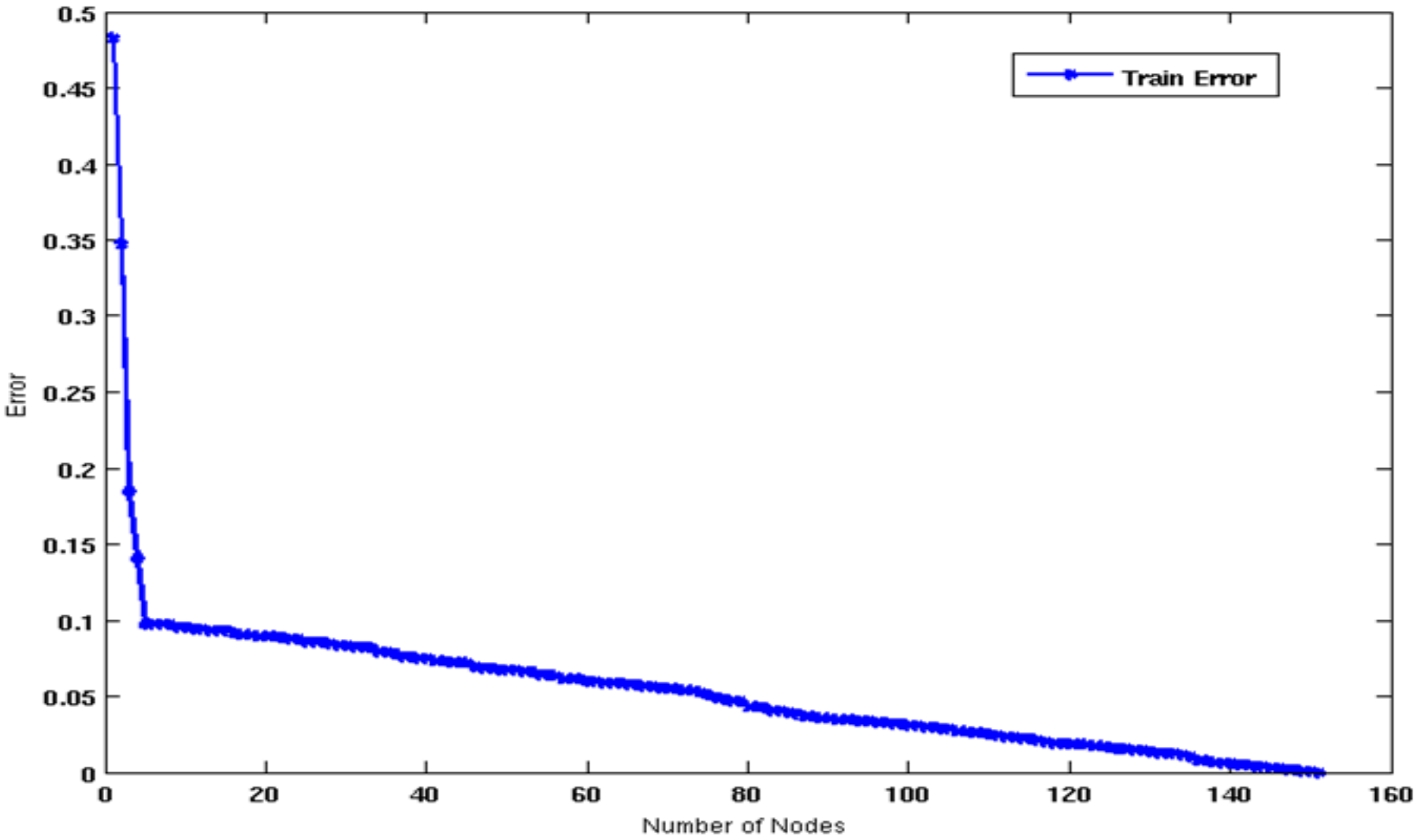
- **5000 instances generated from a Gaussian centered at (10,10)**
- **200 noisy instances added**

o : 5200 instances

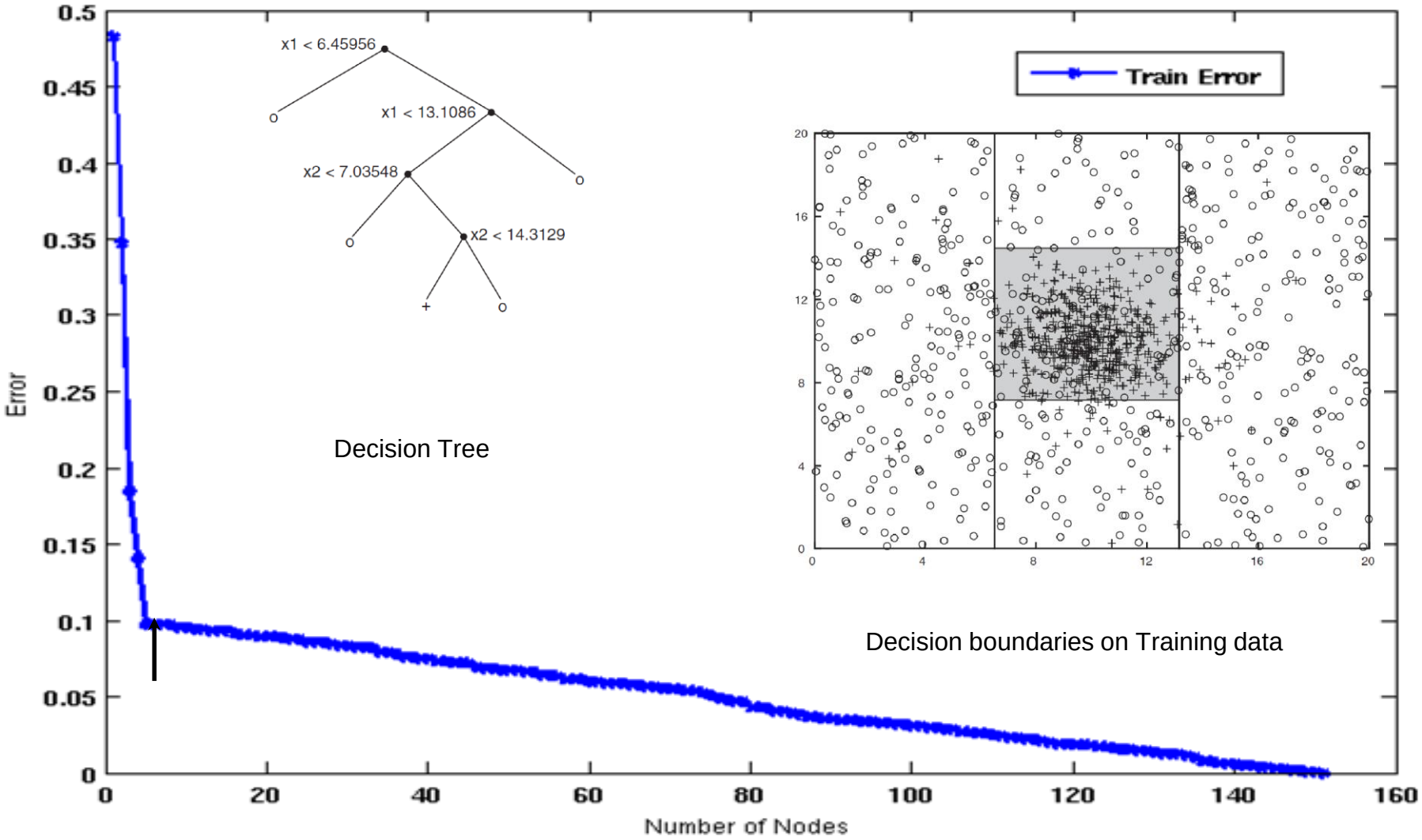
- **Generated from a uniform distribution**

10 % of the data used for training and 90% of the data used for testing

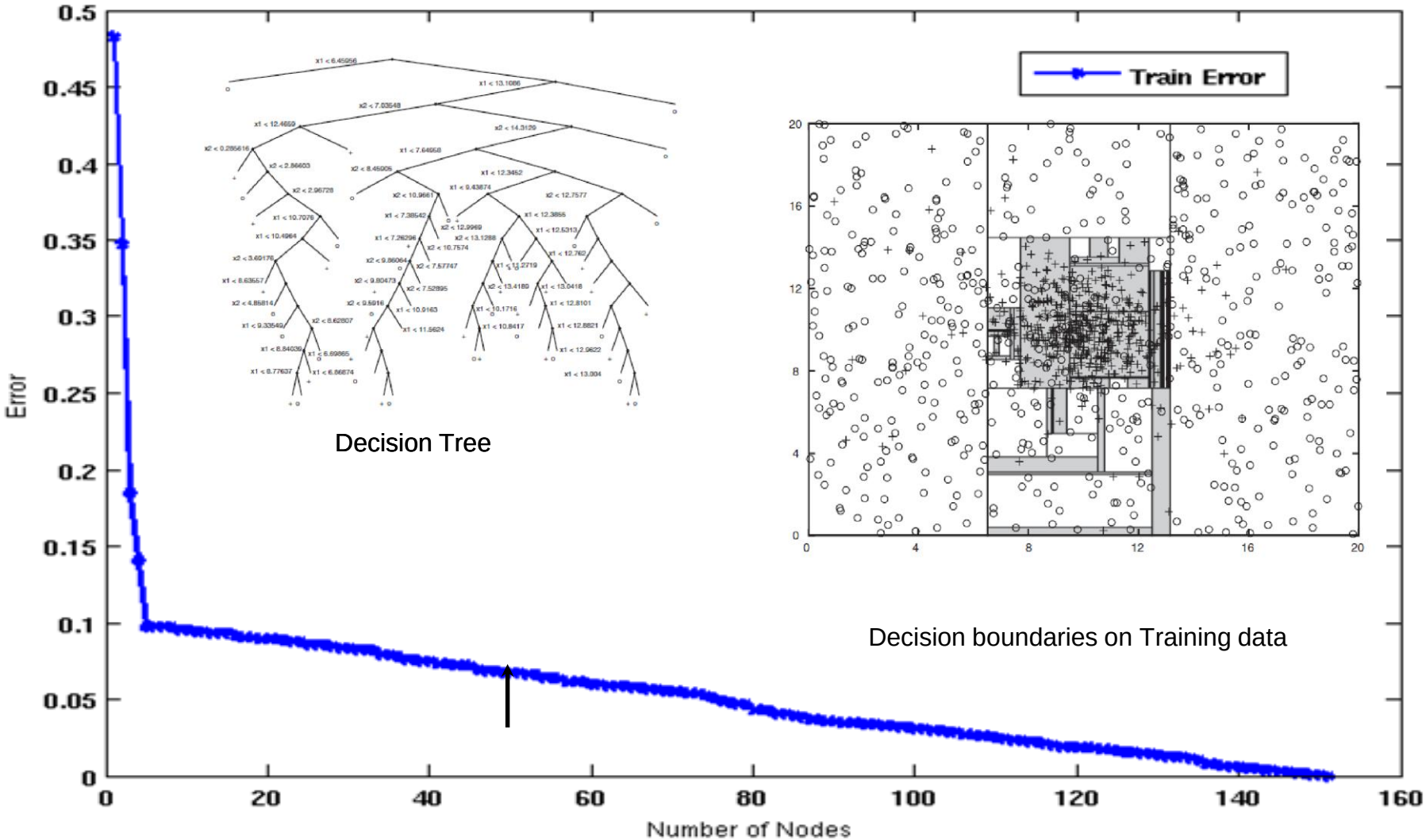
Increasing number of nodes in Decision Trees



Decision Tree with 4 nodes

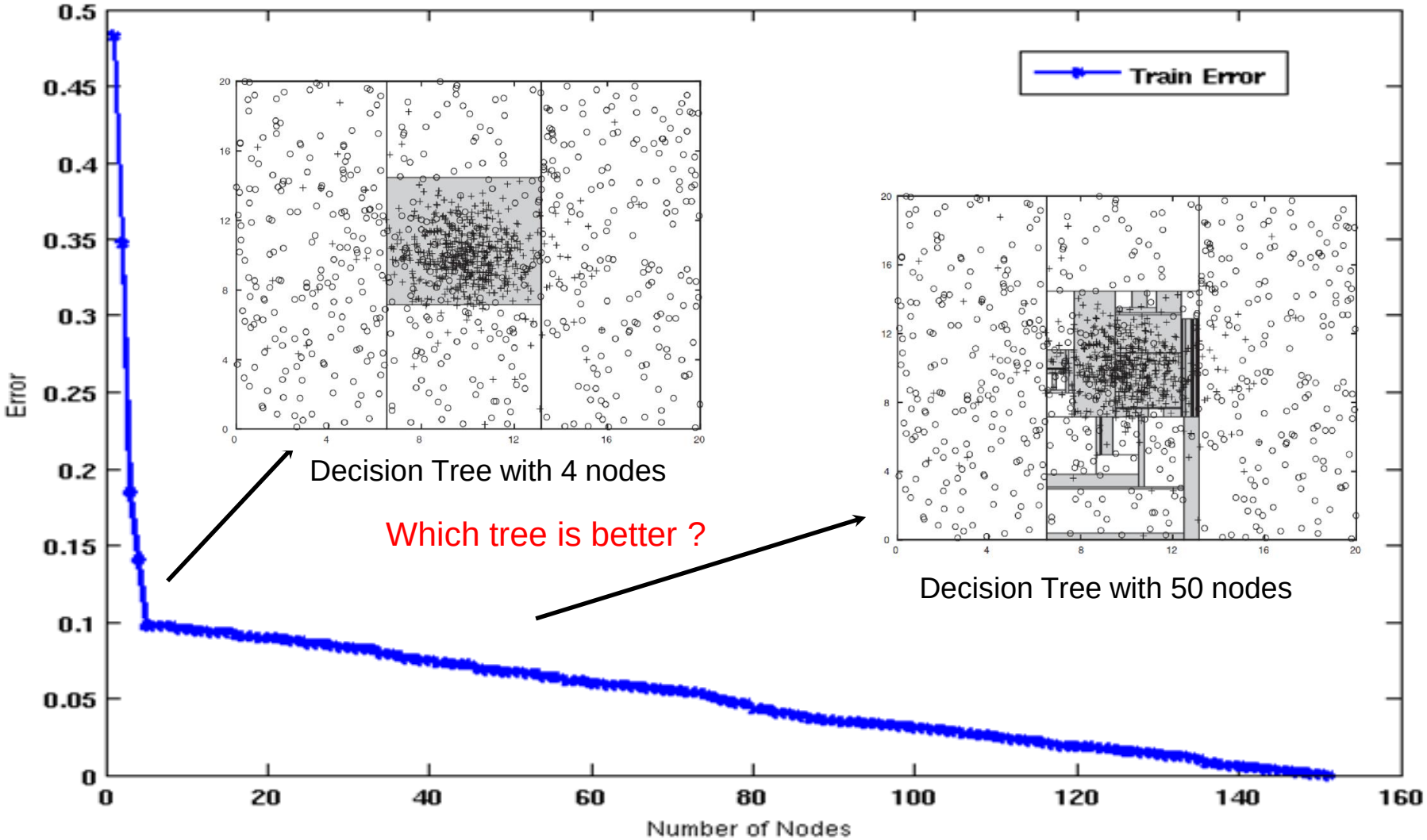


Decision Tree with 50 nodes



Which tree is better?

Problem Type 9 – Model Comparison

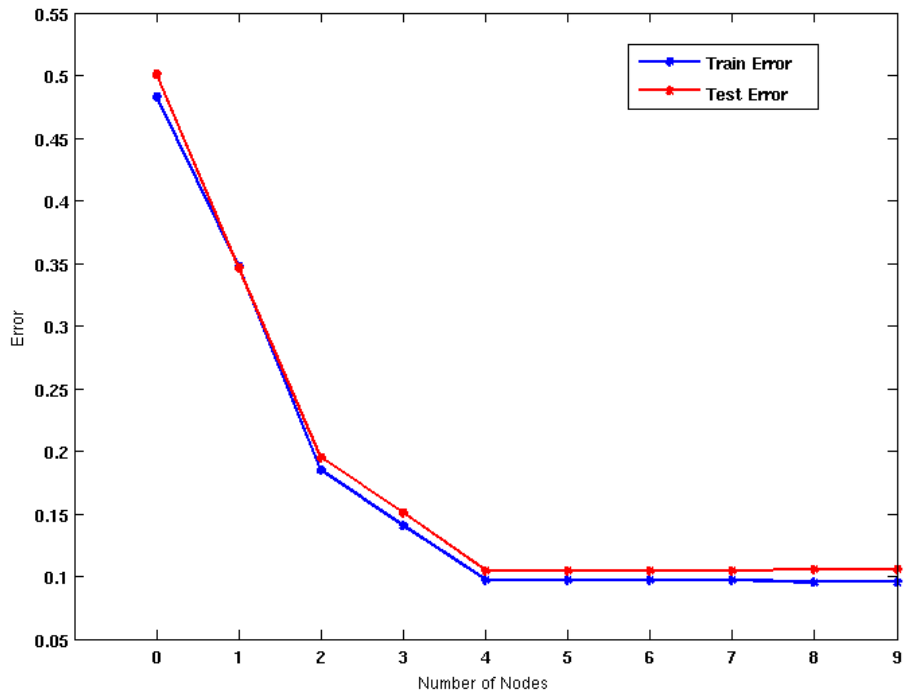


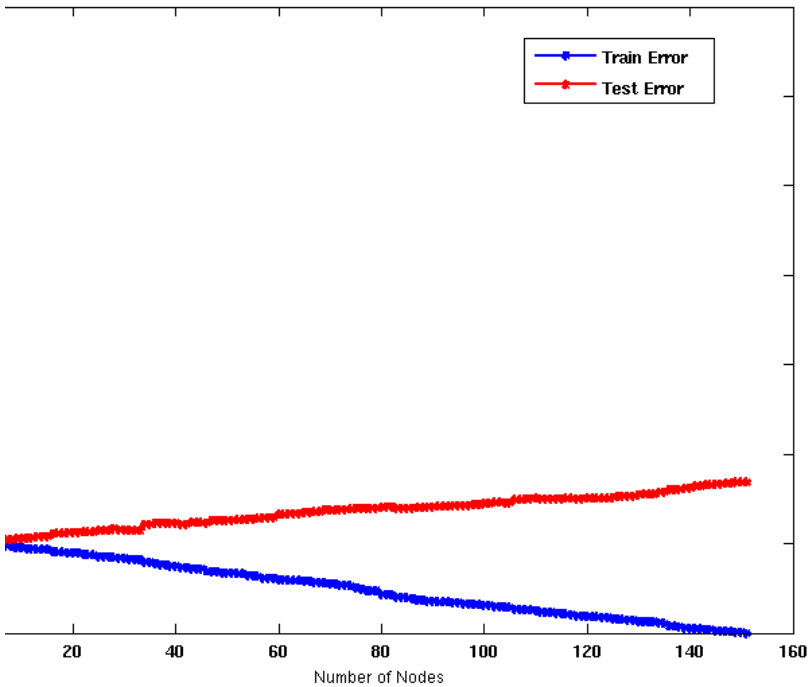
Decision Tree with 4 nodes

Which tree is better ?

Decision Tree with 50 nodes

Model Overfitting



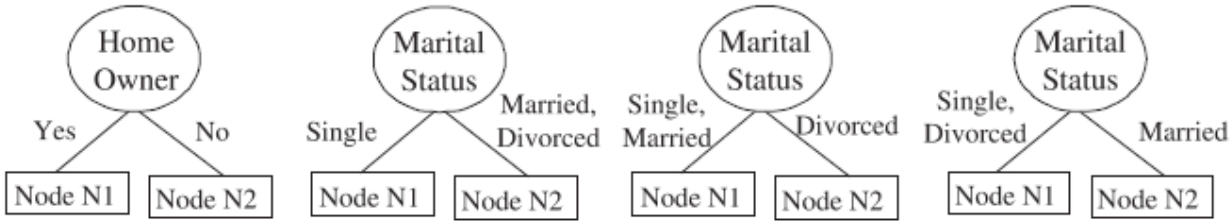


Underfitting: when model is too simple, both training and test errors are large

Overfitting: when model is too complex, training error is small but test error is large

Alternative to Entropy:
Gini Index : Binary Splits

ID	Home Owner	Marital Status	Annual Income	Defaulted?
1	Yes	Single	125000	No
2	No	Married	100000	No
3	No	Single	70000	No
4	Yes	Married	120000	No
5	No	Divorced	95000	Yes
6	No	Single	60000	No
7	Yes	Divorced	220000	No
8	No	Single	85000	Yes
9	No	Married	75000	No
10	No	Single	90000	Yes



Alternative to Entropy:
Gini Index : Binary Splits

Gini Index:Parent

$$Gini\ Index = 1 - \sum_{i=0}^{c-1} p_i(t)^2$$

ID	Home Owner	Marital Status	Annual Income	Defaulted?
1	Yes	Single	125000	No
2	No	Married	100000	No
3	No	Single	70000	No
4	Yes	Married	120000	No
5	No	Divorced	95000	Yes
6	No	Single	60000	No
7	Yes	Divorced	220000	No
8	No	Single	85000	Yes
9	No	Married	75000	No
10	No	Single	90000	Yes

$$1 - \left(\frac{3}{10}\right)^2 - \left(\frac{7}{10}\right)^2 = 0.420.$$

	Parent
No	**7**
Yes	**3**
Gini = 0.420	

Splitting Attribute: HomeOwner

ID	Home Owner	Marital Status	Annual Income	Defaulted?
1	Yes	Single	125000	No
2	No	Married	100000	No
3	No	Single	70000	No
4	Yes	Married	120000	No
5	No	Divorced	95000	Yes
6	No	Single	60000	No
7	Yes	Divorced	220000	No
8	No	Single	85000	Yes
9	No	Married	75000	No
10	No	Single	90000	Yes

$$Gini\ Index = 1 - \sum_{i=0}^{c-1} p_i(t)^2$$

Gini index for the child nodes

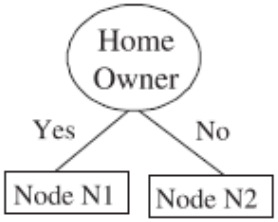
N1 = 1- $(0/3)^2$ - $(3/3)^2$ = 0

N2 =1- $(3/7)^2$ - $(4/7)^2$ = 0.490

Weighted Gini Index for children

$$I(\text{children}) = \sum_{j=1}^{k} \frac{N(v_j)}{N} I(v_j)$$

$$(3/10) \times 0 + (7/10) \times 0.490 = 0.343,$$



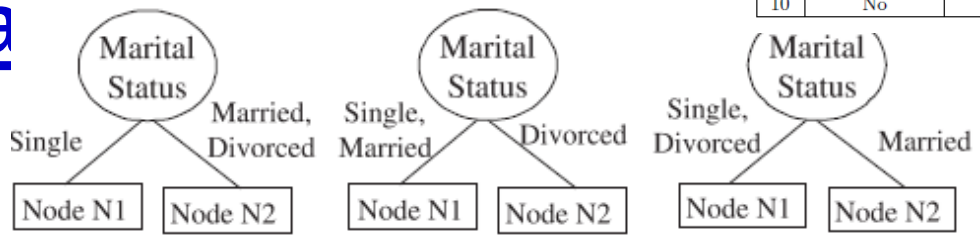
	N1	N2
No	3	4
Yes	0	3
Gini= 0.343		

The gain using Home Owner as splitting attribute is 0.420 - 0.343 =0.077

Alternative to Entropy:
Gini Index : Binary Splits

ID	Home Owner	Marital Status	Annual Income	Defaulted?
1	Yes	Single	125000	No
2	No	Married	100000	No
3	No	Single	70000	No
4	Yes	Married	120000	No
5	No	Divorced	95000	Yes
6	No	Single	60000	No
7	Yes	Divorced	220000	No
8	No	Single	85000	Yes
9	No	Married	75000	No
10	No	Single	90000	Yes

Splitting Attribute: Marita



	N1	N2
No	3	4
Yes	2	1
Gini= 0.400		

	N1	N2
No	6	1
Yes	2	1
Gini= 0.400		

	N1	N2
No	4	3
Yes	3	0
Gini= 0.343		

Based on these results, Home Owner and the last binary split using Marital Status are clearly the best candidates, since they both produce the lowest weighted average Gini index

An alternative measure: gain ratio

$$SplitInformation(S, A) \equiv -\sum_{i=1}^{c} \frac{|S_i|}{|S|} \log_2 \frac{|S_i|}{|S|}$$

- S_i are the sets obtained by partitioning on value i of A
- *SplitInformation* measures the entropy of S with respect to the values of A. The more uniformly dispersed the data the higher it is.

$$GainRatio(S, A) \equiv \frac{Gain(S, A)}{SplitInformation(S, A)}$$

- *GainRatio* penalizes attributes that split examples in many small classes such as *Date.* Let $|S|=n$, *Date* splits examples in n classes
  - $SplitInformation(S, Date) = -[(1/n \log_2 1/n) + \ldots + (1/n \log_2 1/n)] = -\log_2 1/n = \log_2 n$
- Compare with A, which splits data in two even classes:
  - $SplitInformation(S, A) = -[(1/2 \log_2 1/2) + (1/2 \log_2 1/2)] = -[-1/2 - 1/2] = 1$

Limitation of Information Gain: Ways to tackle

SplitInformation measures the entropy of S with respect to the values of A. The more uniformly dispersed the data the higher it is.

$$SplitInformation(S, A) \equiv -\sum_{i=1}^{c} \frac{|S_i|}{|S|} \log_2 \frac{|S_i|}{|S|}$$

$SplitInformation(S, customerID) = -[(1/n \log_2 1/n) + \ldots + (1/n \log_2 1/n)] = -\log_2 1/n = \log_2 n$

GainRatio penalizes attributes that split examples in many small classes such as ID by incorporating split information.

$$GainRatio(S, A) \equiv \frac{Gain(S, A)}{SplitInformation(S, A)}$$

Customer ID	Gender	Car Type	Shirt Size	Class
1	M	Family	Small	C0
2	M	Sports	Medium	C0
3	M	Sports	Medium	C0
4	M	Sports	Large	C0
5	M	Sports	Extra Large	C0
6	M	Sports	Extra Large	C0
7	F	Sports	Small	C0
8	F	Sports	Small	C0
9	F	Sports	Medium	C0
10	F	Luxury	Large	C0
11	M	Family	Large	C1
12	M	Family	Extra Large	C1
13	M	Family	Medium	C1
14	M	Luxury	Extra Large	C1
15	F	Luxury	Small	C1
16	F	Luxury	Small	C1
17	F	Luxury	Medium	C1
18	F	Luxury	Medium	C1
19	F	Luxury	Medium	C1
20	F	Luxury	Large	C1

Select the best attribute test condition among the following three attributes: **Gender, Car Type, and Customer ID**

Limitation of Information Gain: Ways to tackle

Problem Type 10

$$\text{Entropy(parent)} = -\frac{10}{20}\log_2\frac{10}{20} - \frac{10}{20}\log_2\frac{10}{20} = 1.$$

If `Gender` is used as attribute test condition:

$$\text{Entropy(children)} = \frac{10}{20}\left[-\frac{6}{10}\log_2\frac{6}{10} - \frac{4}{10}\log_2\frac{4}{10}\right] \times 2 = 0.971$$

$$\text{Gain Ratio} = \frac{1 - 0.971}{-\frac{10}{20}\log_2\frac{10}{20} - \frac{10}{20}\log_2\frac{10}{20}} = \frac{0.029}{1} = 0.029$$

If `Car Type` is used as attribute test condition:

$$\text{Entropy(children)} = \frac{4}{20}\left[-\frac{1}{4}\log_2\frac{1}{4} - \frac{3}{4}\log_2\frac{3}{4}\right] + \frac{8}{20} \times 0 + \frac{8}{20}\left[-\frac{1}{8}\log_2\frac{1}{8} - \frac{7}{8}\log_2\frac{7}{8}\right] = 0.380$$

$$\text{Gain Ratio} = \frac{1 - 0.380}{-\frac{4}{20}\log_2\frac{4}{20} - \frac{8}{20}\log_2\frac{8}{20} - \frac{8}{20}\log_2\frac{8}{20}} = \frac{0.620}{1.52} = 0.41$$

Customer ID	Gender	Car Type	Shirt Size	Class
1	M	Family	Small	C0
2	M	Sports	Medium	C0
3	M	Sports	Medium	C0
4	M	Sports	Large	C0
5	M	Sports	Extra Large	C0
6	M	Sports	Extra Large	C0
7	F	Sports	Small	C0
8	F	Sports	Small	C0
9	F	Sports	Medium	C0
10	F	Luxury	Large	C0
11	M	Family	Large	C1
12	M	Family	Extra Large	C1
13	M	Family	Medium	C1
14	M	Luxury	Extra Large	C1
15	F	Luxury	Small	C1
16	F	Luxury	Small	C1
17	F	Luxury	Medium	C1
18	F	Luxury	Medium	C1
19	F	Luxury	Medium	C1
20	F	Luxury	Large	C1

Limitation of Information Gain: Ways to tackle

Problem Type 10

Finally, if `Customer ID` is used as attribute test condition:

$$\text{Entropy(children)} = \frac{1}{20}\left[-\frac{1}{1}\log_2\frac{1}{1} - \frac{0}{1}\log_2\frac{0}{1}\right] \times 20 = 0$$

$$\text{Gain Ratio} = \frac{1-0}{-\frac{1}{20}\log_2\frac{1}{20} \times 20} = \frac{1}{4.32} = 0.23$$

Customer ID	Gender	Car Type	Shirt Size	Class
1	M	Family	Small	C0
2	M	Sports	Medium	C0
3	M	Sports	Medium	C0
4	M	Sports	Large	C0
5	M	Sports	Extra Large	C0
6	M	Sports	Extra Large	C0
7	F	Sports	Small	C0
8	F	Sports	Small	C0
9	F	Sports	Medium	C0
10	F	Luxury	Large	C0
11	M	Family	Large	C1
12	M	Family	Extra Large	C1
13	M	Family	Medium	C1
14	M	Luxury	Extra Large	C1
15	F	Luxury	Small	C1
16	F	Luxury	Small	C1
17	F	Luxury	Medium	C1
18	F	Luxury	Medium	C1
19	F	Luxury	Medium	C1
20	F	Luxury	Large	C1

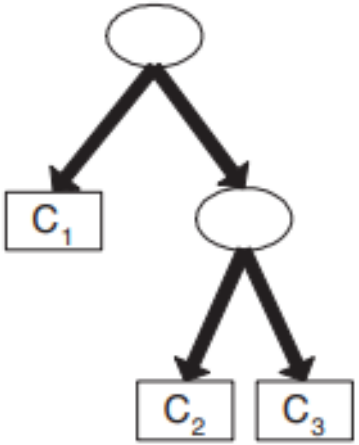
Inference : Thus, even though Customer ID has the highest information gain, its gain ratio is lower than Car Type since it produces a larger number of splits

Minimum Description Length Principle

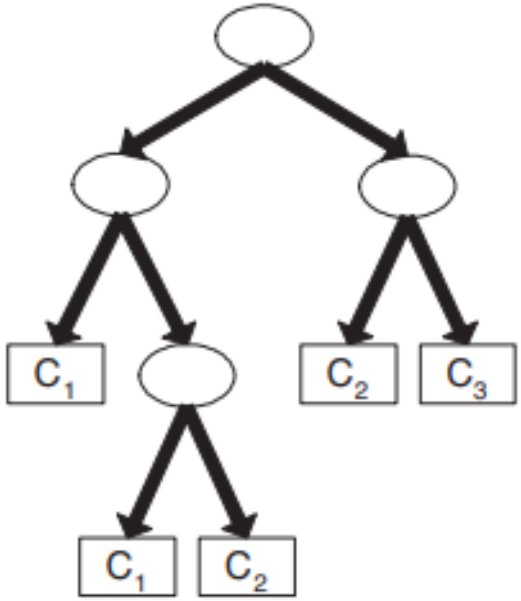
Problem Type 11

Consider the decision trees shown in Figure 3. Assume they are generated from a data set that contains 16 binary attributes and 3 classes, C_1, C_2, and C_3. Compute the total description length of each decision tree according to the minimum description length principle.

Which decision tree is better, according to the MDL principle?



(a) Decision tree with 7 errors



(b) Decision tree with 4 errors

Minimum Description Length Principle

Problem Type 11

Answer:

Because there are 16 attributes, the cost for each internal node in the decision tree is:

$$\log_2(m) = \log_2(16) = 4$$

Furthermore, because there are 3 classes, the cost for each leaf node is:

$$\lceil \log_2(k) \rceil = \lceil \log_2(3) \rceil = 2$$

The cost for each misclassification error is $\log_2(n)$.

The overall cost for the decision tree (a) is $2\times4+3\times2+7\times\log_2 n = 14+7\log_2 n$ and the overall cost for the decision tree (b) is $4\times4+5\times2+4\times5 = 26+4\log_2 n$. According to the MDL principle, tree (a) is better than (b) if $n < 16$ and is worse than (b) if $n > 16$.

Minimum Description Length Principles

$$
\begin{aligned}
h_{MAP} &= \arg\max_{h \in H} P(D|h)P(h) \\
&= \arg\max_{h \in H} \log_2 P(D|h) + \log_2 P(h) \\
&= \arg\min_{h \in H} -\log_2 P(D|h) - \log_2 P(h) \quad (1)
\end{aligned}
$$

Interesting fact from information theory:

The optimal (shortest expected coding length) code for an event with probability p is $-\log_2 p$ bits.

So interpret (1):

- $L_{C1}(h) = length(h) = -\log_2 P(h)$ [cost to encode the model]
- $L_{C2}(D|h) = length(misclassifications) = -\log_2 P(D|h)$ [cost to encode the data errors given the model]

→ prefer the hypothesis that minimizes

$$length(h) + length(misclassifications)$$

Minimum Description Length Principle

- MDL principle provides a way of trading off hypothesis complexity for the number of errors committed by the hypothesis.
- May select a shorter hypothesis that makes a few errors over a longer hypothesis that perfectly classifies the training data.
- Provides one method for dealing with the issue of overfitting the data.

Work Integrated Learning Programmes

Minimum Description Length Principle

Occam's razor: prefer the shortest hypothesis

MDL: prefer the hypothesis *h* that minimizes

$$h_{MDL} = \underset{h \in H}{\operatorname{argmin}}\, L_{C_1}(h) + L_{C_2}(D|h)$$

where $L_C(x)$ is the description length of *x* under encoding *C*

Example: H = decision trees hypothesis, D = training data labels

- $L_{C1}(h)$ is # bits to describe tree *h*
- $L_{C2}(D|h)$ is # bits to describe *D* given *h*
  - Note $L_{C2}(D|h) = 0$ if examples classified perfectly by *h*. Need only describe exceptions
- Hence h_{MDL} trades off tree size for training errors